Comparing Rows with SQL

Lecture 3: Rankings, changes, and running totals

Gregory S. DeLozier, PhD

Today

Calculation	Dashboard result
dense_rank	Rating levels within each year
lag	Change from a preceding yearly row
sum over a frame	Running movie count

Exercise: Run the Small Examples

```
source .venv/bin/activate
export DATABASE_URL=postgresql://student@127.0.0.1:5433/imdb
psql "$DATABASE_URL" -f lecture-03-windows.sql
```

The VALUES examples run without changing stored data.

GROUP BY and OVER

GROUP BY	Window function with OVER
One output row per group	One output row per input row
Summarize a year	Compare a movie with its year
count, avg	dense_rank, lag, windowed sum

Rank Within Each Year

```
dense_rank() OVER (
    PARTITION BY t.start_year
    ORDER BY r.average_rating DESC
) AS year_rank
```

Partition selects the group. Window ordering selects rank order.

Ties Are Part of the Result

Film	Rating	dense_rank	row_number
Alpha	8.0	1	1
Beta	8.0	1	2
Gamma	7.0	2	3

Top three rating levels can contain more than three films.

Filter After Ranking

```
WITH ranked AS (
  SELECT t.primary_title,
         dense_rank() OVER (
           PARTITION BY start_year
           ORDER BY average_rating DESC) AS year_rank
  FROM imdb_course.titles t
  JOIN imdb_course.ratings r USING (title_id)
)
SELECT * FROM ranked WHERE year_rank <= 3;
```

Previous Row Is Not Always Previous Year

Year	Movies	Previous year	Change
2000	3	NULL	NULL
2002	1	2000	-2

lag follows the rows present in the result.

A Running Total Has a Frame

```
sum(movies) OVER (
    ORDER BY start_year
    ROWS BETWEEN UNBOUNDED PRECEDING
             AND CURRENT ROW
) AS running_total
```

Counts 3 and 1 produce running totals 3 and 4.

Exercise: Open the Extended App

```
streamlit run app.py --server.address 0.0.0.0
```

Run lecture-03-windows.sql to inspect shared ranks.

Reports: cumulative counts and yearly differences.

Changing the vote threshold changes the population.

Exercise: Exactly Three Movies

File: `database.py`, function: `ranked_movies`

Use `row_number` with rating, votes, and title identifier in its ordering.

Keep rows numbered 1 through 3 within each year.

Expected: at most three qualifying movies per year.

Exercise: Each Movie and Its Year

Add `avg(average_rating) OVER (PARTITION BY start_year)`.

Display the movie rating, its year mean, and the difference.

Expected: movie rows remain visible beside their comparison value.

From Files to an Analytic Application

Import once. Verify the data.

Express each calculation in SQL.

Keep query logic in the data layer.

Use the dashboard to inspect what the query actually means.